

西安交通大学

课程实验报告

实验名称: 数据挖掘分类算法实践

组 长: 林圣翔

组 员: 易怀川、李彦铨

报告日期: 2025.9.30



一、实验目的

1. 系统掌握经典数据挖掘算法的核心原理与实现。
2. 培养模型构建、调优与评估的完整工程实践技能。
3. 提升针对现实场景的建模能力与解决方案设计水平。

二、实验内容

1. 完成多种分类算法的完整应用流程，包括数据处理、模型训练与综合评估。
2. 针对 K 近邻 (KNN)、随机森林 (Random Forest) 及朴素贝叶斯 (Naive Bayes) 三个关键算法，在不依赖 scikit-learn 等高级机器学习库的前提下，完成从零开始的手动实现。
3. 对代码进行工程化重构，强调模块化与接口设计，确保系统具备良好的可扩展性。

三、实验环境

1. 系统环境：基于 Ubuntu 20.04.6 LTS 版本
2. 编程语言：Python 3.10

四、数据集简介

本次实验所采用的数据集来源于 uci 等主流的数据科学网站，在实验过程中我们采用了如下几个数据集：

数据集	特征数量	类别数量	训练样本数	测试样本数
kdd_synthetic_control	60	6	480	120
mfeat-pixel	240	10	1600	400
optdigits	64	10	3060	765

kdd_synthetic_control：这是一个合成生成的时间序列数据集，专门用于模拟工业生产中的质量控制图。数据集中包含了 600 条控制图表，每条时间序列有 60 个数据点。用于时间序列分类、模式识别和异常检测的研究。任务是训练模型识别时间序列属于哪种控制图模式。

mfeat-pixel：这是一个非常全面的手写数字特征数据集，包含从荷兰公用事业地图中提取的手写数字（0-9）的多种特征表示。它的独特之处在于，每个数字样本都由 6 种不同的特征集进行描述，存储在不同的文件中。用于多特征模式识别、分类器融合和比较不同特征表示的有效性。研究如何结合来自不同特征源的信息来构建更强大的分类器。

optdigits：由 Bogazici 大学计算机工程系的 E. Alpaydin 和 C. Kaynak

等人构建，包含了手写数字（0-9）的图像数据。每个数字图像通过 8x8 的像素矩阵表示。用于手写数字识别、图像分类和机器学习算法的基准测试。

五、算法原理

1. 分类算法

1.1. KNN 算法

KNN（K-近邻）算法是一种基于实例的监督学习方法，广泛应用于分类与回归任务。该算法的基本思想是：对于一个新的数据点，首先在训练集中找出与其最相似的 K 个样本，然后根据这些近邻样本的标签进行“多数投票”（用于分类）或计算“平均值”（用于回归），从而确定新数据点的输出结果。KNN 方法的核心在于数据点之间的距离度量，以及基于最近邻的决策机制。

实验中自定义 KNN 实现采用了基于暴力搜索的核心算法思路，通过简单存储训练数据的 fit 方法和计算每个测试样本与所有训练样本多种距离度量的预测流程，结合 k 近邻的权重投票策略完成分类任务，其训练时间复杂度为 $O(1)$ ，预测时间复杂度高达 $O(m \times n \times d)$ 。

1.2. Randomforest 算法

随机森林是一种基于 Bagging 框架的集成学习方法，其基学习器为决策树，多棵决策树共同构成“森林”。该算法的“随机性”主要体现在两个方面：首先，在样本选择上采用自助采样引入样本扰动；其次，在每棵决策树的节点分裂时，会从全部候选特征中随机选取一个包含 K 个特征的子集，并仅在该子集中选择最优划分属性。这种双重随机机制有效提升了模型的多样性与泛化能力。

实验中自定义决策树与随机森林实现基于 CART 算法构建，决策树采用递归分裂策略通过基尼不纯度评估特征重要性并选择最佳分裂点，支持随机特征选择和多种停止条件；随机森林则通过自助采样和特征随机性构建多棵决策树，最终通过投票机制进行集成预测。在复杂度方面，决策树训练最坏情况下训练时间复杂度为 $O(n \times d \times \log(n) \times c)$ ，预测时间复杂度为 $O(\log(n) \times t)$ ，随机森林则在此基础上乘以树的数量因子。

1.3. Naivebayes 算法

朴素贝叶斯分类方法基于贝叶斯定理，通过特征属性的先验概率计算其后验概率，并选择后验概率最大的类别作为该样本的预测结果。该方法被称为“朴素”，因为它基于一个简单而基本的假设：所有特征在给定类别条件下相互独立。尽管该假设在现实中往往不成立，但朴素贝叶斯仍能在许多任务中表现出良好的分类性能。

实验中自定义朴素贝叶斯实现通过分别处理高斯和多项式两种分布假设来完成分类任务，采用基于概率统计的参数估计方法，在高斯分布中计算每个类别的特征均值和方差并使用高斯概率密度函数进行似然估计，在多项式分布中则通过特征计数和拉普拉斯平滑计算条件概率，其训练时间复杂度为 $O(n \times d \times c)$ ，预测时间复杂度为 $O(t \times d \times c)$ 。

2. 评测指标

2.1. 精度

精度是指在所有被预测为正类的样本中，实际为正类的样本所占的比例，用于衡量模型预测正类的准确性。

2.2. 召回率

召回率是指在所有实际为正类的样本中，被模型正确预测为正类的样本所占的比例，用于评估模型对正类样本的识别能力。

2.3. F1 分数

F1 分数是精度和召回率的调和平均数，综合反映模型的整体分类性能，尤其适用于不平衡数据集。

2.4. 混淆矩阵

混淆矩阵以矩阵形式展示分类模型在不同类别上的预测结果与真实标签的对应情况，是计算精度、召回率等指标的基础工具。

2.5. 袋外分数

在基于自助采样法构建模型时，每次采样后都会剩余部分未被选中的样本，即袋外样本。袋外分数是指利用这些未被采样的数据作为验证集，对模型性能进行评估所得的分数，常用于随机森林等集成方法中，作为模型泛化能力的内在估计。

六、实验过程

1. 环境配置

1.1. 创建 Conda 虚拟环境

```
conda create -n DM python=3.10
```

1.2. 激活 Conda 虚拟环境

```
conda activate DM
```

1.3. 安装项目依赖包

```
pip install -r requirements.txt
```

2. 代码实现

2.1. run.py

2.1.1. 数据预处理模块

系统实现了完整的数据预处理流水线，支持 ARFF 格式数据集，包含以下关键处理步骤：

(1) 数据加载与清洗

①ARFF 格式解析：使用 `scipy.io.arff` 库专门处理 ARFF 格式数据集。

②目标变量自动检测：系统自动识别数据集的最后一列作为目标变量。

③数据类型智能转换：字符串特征自动转换为数值型，分类变量使用 `LabelEncoder` 进行编码，数值型特征保持原始格式等。

(2) 预处理流水线优化

系统采用 `scikit-learn` 的 Pipeline 机制，构建了标准化的预处理流程：

```
pipeline_steps = [  
    ('imputer', SimpleImputer(strategy='most_frequent')),      # 缺失值处理  
    ('variance_threshold', VarianceThreshold(threshold=0.01)), # 低方差特征过滤  
    ('selector', SelectKBest(k=min(9, initial_features))),     # 特征选择  
]
```

特殊算法适配处理：

- ①对 SVM、KNN 等距离敏感算法自动添加标准化步骤。
- ②聚类算法进行特征缩放以保证距离计算的有效性。
- ③树模型算法保持原始特征分布。

2.1.2. 模型训练

(1) 多算法集成框架

集成包括 C4.5、CART 等在内的 8 种分类算法，其中选取 RandomForest、KNN、NaiveBayes 这三个算法进行了自定义实现。

算法类别	算法名称	实现方式	特点
决策树	C4.5	Orange3 库封装	信息增益分裂
决策树	CART	scikit-learn	Gini 系数分裂
集成学习	RandomForest	scikit-learn	原生随机森林
集成学习	Custom_RandomForest	自定义实现	支持袋外评估
集成学习	AdaBoost	scikit-learn	自适应增强
邻近算法	KNN	scikit-learn	K 近邻分类
邻近算法	Custom_KNN	自定义实现	距离度量优化
贝叶斯	NaiveBayes	scikit-learn	高斯朴素贝叶斯
贝叶斯	Custom_NaiveBayes	自定义实现	多分布支持
支持向量机	SVM	scikit-learn	核方法分类
聚类算法	K-Means	包装为分类器	无监督转有监督

(2) 自动化超参数优化：采用网格搜索(GridSearchCV) 技术进行参数调优：

- ①搜索策略：穷举式参数网格搜索
- ②验证方法：5 折交叉验证
- ③评估指标：分类准确率
- ④并行计算：多核并行加速搜索过程

参数配置示例（以随机森林为例）：

```
param_grid = {
    'n_estimators': [50, 100, 150],      # 树的数量
    'max_depth': [5, 10, 15, 20, None],  # 树的最大深度
    'min_samples_split': [2, 5],          # 分裂最小样本数
    'max_features': ['auto', 'sqrt']      # 特征选择方式
}
```

2.1.3. 评估模块

建立了全面的模型评估框架，从多个角度评估算法性能。

(1) 核心评估指标

- ①训练集准确率：反映模型对训练数据的拟合程度。
- ②测试集准确率：评估模型在未见数据上的泛化能力。
- ③5 折交叉验证：提供稳定可靠的性能估计。
- ④混淆矩阵分析：详细展示分类结果的分布情况。
- ⑤分类报告：精确率、召回率、F1 分数的综合评估。

(2) 附加评估

- ①Custom_RandomForest：袋外分数(OOB Score)评估。
- ②Custom_KNN：距离度量和权重策略分析。

③Custom_NaiveBayes: 不同分布假设的效果比较结果处理与保存模块。

(3) 支持多格式结果输出

①文本报告: 详细的训练过程和评估指标。

②JSON 格式: 结构化的结果数据, 便于后续分析。

③可视化图表: 混淆矩阵和交叉验证准确率分布图。

④汇总 CSV: 所有实验结果的集中记录。

⑤文件组织架构:

```
result/
├── {algorithm_name}/          # 算法专属目录
│   ├── confusion_matrix_*     # 混淆矩阵图片
│   ├── cross_val_accuracy_*   # 交叉验证图表
│   ├── {algorithm}_{dataset}*.txt # 文本报告
│   └── {algorithm}_{dataset}*.json # JSON 数据
└── summary.csv                # 实验汇总表
```

2.1.4. 命令接口设计

提供统一的命令行接口: `python run.py --algorithm XX --dataset XX`

2.1.5. 实验可重复性保障

①随机性控制: 所有随机过程使用统一种子(42), 训练测试分割策略固定。

②实验追踪: 通过时间戳记录实验执行的精确时间, 并保存模型最佳参数配置。

③版本管理: 每次实验生成独立的记录文件, 记录调参过程中的所有参数组合, 通过时间戳可以追溯到具体的实验条件。

2.2. models/custom_knn.py

本模块实现了自定义的 KNN 分类算法, 基于 `scikit-learn` 接口标准进行开发。

2.2.1. 距离度量

算法实现了四种常用的距离度量方法, 具体代码如下:

```
def _calculate_distance(self, x1, x2):
    """计算两个样本之间的距离"""
    if self.metric == 'euclidean':
        return np.sqrt(np.sum((x1 - x2) ** 2)) # 欧几里得距离
    elif self.metric == 'manhattan':
        return np.sum(np.abs(x1 - x2)) # 曼哈顿距离
    elif self.metric == 'minkowski':
        return np.sum(np.abs(x1 - x2) ** self.p) ** (1 / self.p) # 闵可夫斯基距离
    elif self.metric == 'cosine':
        dot_product = np.dot(x1, x2)
        norm_x1 = np.linalg.norm(x1)
        norm_x2 = np.linalg.norm(x2)
        return 1 - (dot_product / (norm_x1 * norm_x2)) # 余弦相似度转换
```

2.2.2. 邻居搜索策略

采用暴力搜索算法, 确保找到精确的最近邻, 实现代码如下:

```
def _get_neighbors(self, x):
    """获取 x 的 k 个最近邻"""
    distances = []
    for i, train_sample in enumerate(self.X_train):
        dist = self._calculate_distance(x, train_sample)
        distances.append((i, dist))
    distances.sort(key=lambda x: x[1]) # 按距离排序并返回前 k 个邻居
    return distances[:self.n_neighbors]
```

2.2.3. 权重计算机制

当权重策略为“统一权重”时，所有邻居权重相同，当权重策略为“距离权重”时，以距离的倒数作为权重，同时避免除零错误，实现代码如下：

```
if self.weights == 'uniform':
    weights = np.ones(len(neighbor_labels)) # 所有邻居等权重
else: # distance 权重
    epsilon = 1e-8 # 避免除零错误
    weights = 1.0 / (np.array(neighbor_distances) + epsilon) # 距离倒数作为权重
```

2.2.4. 分类决策策略

①通过加权投票统计各类别得票，确定预测类别，实现代码：

```
# 统计加权投票
vote_counts = {}
for label, weight in zip(neighbor_labels, weights):
    if label not in vote_counts:
        vote_counts[label] = 0
    vote_counts[label] += weight
max_vote = max(vote_counts.values()) # 找到得票最多的类别
candidates = [label for label, count in vote_counts.items() if count == max_vote]
```

②当多个类别得票相同时，通过随机选择打破平局，实现代码：

```
if len(candidates) == 1:
    return candidates[0]
else:
    return np.random.choice(candidates) # 随机打破平局
```

2.2.5. 概率预测实现

实现了概率预测功能，输出每个类别的预测概率（含归一化处理），代码如下：

```
def predict_proba(self, X):
    """预测概率"""
    # 计算每个类别的权重和
    for label, weight in zip(neighbor_labels, weights):
        proba[i, label] += weight
    if np.sum(proba[i]) > 0:
```

```

        proba[i] /= np.sum(proba[i]) # 归一化概率
    else:
        # 如果没有邻居，均匀分布
        proba[i] = np.ones(self.n_classes_) / self.n_classes_

```

2.2.6. 参数验证系统

设置完整的参数验证机制，确保输入参数合法有效，代码如下：

```

def _check_params(self):
    """检查参数有效性"""
    if self.n_neighbors <= 0:
        raise ValueError("n_neighbors must be positive")
    if self.weights not in ['uniform', 'distance']:
        raise ValueError("weights must be 'uniform' or 'distance'")
    if self.metric not in ['euclidean', 'manhattan', 'minkowski', 'cosine']:
        raise ValueError("metric not supported")

```

2.2.7. 设置参数接口

```

def set_params(self, **parameters):
    """设置模型参数（用于网格搜索）"""
    for parameter, value in parameters.items():
        setattr(self, parameter, value)
    return self

```

2.3. models/custom_randomforest.py

2.3.1. 树节点数据结构

定义 `TreeNode` 类存储决策树的节点信息，包含分裂特征、阈值、子树引用、叶节点预测值等关键属性：

```

class TreeNode:
    """决策树节点类（CART 算法专用）"""
    def __init__(self, feature_index=None, threshold=None, left=None, right=None,
                 value=None, gini=None, samples=None):
        self.feature_index = feature_index # 分裂时使用的特征索引（非叶节点有效）
        self.threshold = threshold         # 分裂阈值（非叶节点有效，如“特征 3≤5.2”）
        self.left = left                   # 左子树引用（特征值≤阈值的样本走向）
        self.right = right                  # 右子树引用（特征值>阈值的样本走向）
        self.value = value                  # 叶节点的预测类别（叶节点有效）
        self.gini = gini                   # 节点的基尼不纯度（衡量节点纯度）
        self.samples = samples              # 节点包含的样本数量（用于权重计算）

```

2.3.2. CART 算法核心机制

(1) 基尼不纯度计算：衡量节点样本的类别混杂程度（值越小，纯度越高）

```

def _gini_impurity(self, y):
    """计算样本集 y 的基尼不纯度（CART 算法分裂标准）"""
    if len(y) == 0: # 空样本集，纯度最高（基尼不纯度为 0）

```

```

        return 0

    class_counts = np.bincount(y) # 统计每个类别的样本数量
    class_probs = class_counts / len(y) # 计算每个类别的概率
    return 1 - np.sum(class_probs ** 2) # 基尼不纯度公式

```

(2) 最佳分裂点搜索：对随机选择的特征子集，遍历每个特征的所有可能阈值，找到“加权基尼不纯度最小”的分裂点：

```

def _find_best_split(self, X, y):
    """寻找最佳分裂特征和阈值（基于基尼不纯度最小化）"""
    n_samples, n_features = X.shape
    best_gini = float('inf') # 初始化为无穷大（目标：找到更小的基尼不纯度）
    best_feature = None      # 最佳分裂特征索引
    best_threshold = None    # 最佳分裂阈值

    # 1. 获取随机特征子集（随机森林的“特征随机性”体现）
    feature_indices = self._get_feature_subset(n_features)

    # 2. 遍历每个候选特征
    for feature_idx in feature_indices:
        feature_values = X[:, feature_idx] # 当前特征的所有样本值
        unique_values = np.unique(feature_values) # 去重，减少分裂点候选

        # 3. 遍历每个候选阈值（特征的唯一值）
        for threshold in unique_values:
            # 划分左右子树样本（特征值≤阈值→左，否则→右）
            left_mask = feature_values <= threshold
            right_mask = ~left_mask
            n_left = np.sum(left_mask)
            n_right = np.sum(right_mask)

            # 跳过空节点（无样本的子树无意义）
            if n_left == 0 or n_right == 0:
                continue

            # 4. 计算分裂后的加权基尼不纯度
            left_gini = self._gini_impurity(y[left_mask])
            right_gini = self._gini_impurity(y[right_mask])

            # 加权：按子树样本数量占比计算总基尼不纯度
            weighted_gini = (n_left * left_gini + n_right * right_gini) / n_samples

            # 5. 更新最佳分裂点（基尼不纯度更小则替换）
            if weighted_gini < best_gini:
                best_gini = weighted_gini
                best_feature = feature_idx
                best_threshold = threshold

    return best_feature, best_threshold, best_gini

```

(3) 递归树构建

通过递归方式构建决策树，当满足“最大深度”“最小样本数”“纯节点”等终止条件时，生成叶节点：

```

def _build_tree(self, X, y, depth=0):
    """递归构建 CART 决策树"""
    n_samples, _ = X.shape
    current_gini = self._gini_impurity(y)
    # 终止条件（满足任一则生成叶节点）
    if (depth >= self.max_depth or # 达到最大深度（防止过拟合）
        n_samples < self.min_samples_split or # 样本数不足分裂（防止过拟合）
        len(np.unique(y)) == 1): # 节点样本全为同一类别（纯节点，无需分裂）
        # 计算叶节点预测值（多数投票：样本数最多的类别）
        leaf_value = self._calculate_leaf_value(y)
        return TreeNode(
            value=leaf_value,
            gini=current_gini,
            samples=n_samples
        )
    # 未满足终止条件：寻找最佳分裂点并递归构建子树
    best_feature, best_threshold, best_gini = self._find_best_split(X, y)
    # 划分左右子树样本
    left_mask = X[:, best_feature] <= best_threshold
    right_mask = ~left_mask
    # 递归构建左、右子树（深度+1）
    left_subtree = self._build_tree(X[left_mask], y[left_mask], depth + 1)
    right_subtree = self._build_tree(X[right_mask], y[right_mask], depth + 1)
    # 返回当前非叶节点（包含分裂信息和子树引用）
    return TreeNode(
        feature_index=best_feature,
        threshold=best_threshold,
        left=left_subtree,
        right=right_subtree,
        gini=best_gini,
        samples=n_samples
    )

def _calculate_leaf_value(self, y):
    """计算叶节点预测值（多数投票：统计样本数最多的类别）"""
    class_counts = np.bincount(y)
    return np.argmax(class_counts) # 返回样本数最多的类别索引

```

2.3.3. 随机森林集成策略

随机森林通过“多棵独立决策树的集成”提升性能，核心是“双重随机性”（样本+特征）和“并行构建”，具体实现如下：

（1）双重随机性实现

①Bootstrap 数据采样：通过“有放回采样”为每棵树生成独立的训练集（Bootstrap 集），未被采样的样本即为“袋外样本”（用于后续评估）：

```
def _bootstrap_sample(self, n_samples, tree_index):
    """Bootstrap 采样：为第 tree_index 棵树生成训练样本索引（有放回）"""
    # 设置随机种子（确保每棵树的采样可复现）
    if self.random_state is not None:
        np.random.seed(self.random_state + tree_index)
    # 有放回采样 n_samples 个样本（索引范围：0~n_samples-1）
    indices = np.random.choice(n_samples, size=n_samples, replace=True)
    return indices
```

②随机特征选择：每棵树训练时仅使用“随机子集”的特征（而非所有特征），减少树之间的相关性，增强集成效果：

```
def _get_max_features(self, n_features):
    """确定每棵树分裂时考虑的特征数量（随机特征子集大小）"""
    if self.max_features == 'auto' or self.max_features == 'sqrt': # 常用策略：特征数的平方根
        return int(np.sqrt(n_features))
    elif self.max_features == 'log2': # 备选策略：特征数的对数（底为 2，适合高维特征）
        return int(np.log2(n_features))
    elif isinstance(self.max_features, int): # 固定数量：不超过总特征数
        return min(self.max_features, n_features)
    elif isinstance(self.max_features, float): # 比例：总特征数的指定比例（如 0.5 表示 50%）
        return int(self.max_features * n_features)
    else: # 默认：使用所有特征（退化为普通决策树集成）
        return n_features
```

（2）森林构建过程

通过循环为每棵树执行“采样→选特征→建树”三步，最终形成随机森林：

```
def fit(self, X, y):
    """训练随机森林：构建 n_estimators 棵独立决策树"""
    X = np.asarray(X)
    y = np.asarray(y)
    n_samples, n_features = X.shape
    # 初始化存储：基学习器（决策树）、每棵树使用的特征索引
    self.estimators_ = []
    self.feature_indices_ = []
    # 记录类别信息（用于后续概率预测）
    self.classes_ = np.unique(y)
    self.n_classes_ = len(self.classes_)
    # 循环构建每棵树
    for i in range(self.n_estimators):
        # 1. Bootstrap 采样：生成当前树的训练样本索引
        indices = self._bootstrap_sample(n_samples, i)
        X_bootstrap = X[indices] # 当前树的训练特征
        y_bootstrap = y[indices] # 当前树的训练标签
```

```

# 2. 随机特征选择：确定当前树使用的特征子集
max_features = self._get_max_features(n_features)
feature_indices = np.random.choice(
    n_features, size=max_features, replace=False
)
# 仅保留选中的特征（减少计算量，增强随机性）
X_bootstrap_subset = X_bootstrap[:, feature_indices]
# 3. 构建自定义决策树（基学习器）
tree = CustomDecisionTree(
    max_depth=self.max_depth,
    min_samples_split=self.min_samples_split,
    min_samples_leaf=self.min_samples_leaf,
    random_state=self.random_state + i if self.random_state else None
)
tree.fit(X_bootstrap_subset, y_bootstrap)
# 4. 保存当前树和特征索引
self.estimators_.append(tree)
self.feature_indices_.append(feature_indices)
# （可选）执行袋外评估（若启用）
if self.oob_score:
    self._compute_oob_score(X, y)
return self

```

2.3.4. 袋外评估机制

袋外样本（未被 **Bootstrap** 采样选中的样本）可作为“天然验证集”，无需额外划分数据即可评估模型性能，实现如下：

（1）袋外样本识别：通过“集合差集”找到每棵树对应的袋外样本

```

def _compute_oob_score(self, X, y):
    """计算袋外（OOB）分数：使用未被采样的样本评估模型"""
    n_samples = X.shape[0]
    # 初始化：存储每个样本的 OOB 预测概率（样本数×类别数）、被预测次数
    oob_predictions = np.zeros((n_samples, self.n_classes_))
    oob_counts = np.zeros(n_samples)
    # 遍历每棵树，收集其对袋外样本的预测
    for i, (tree, feature_indices) in enumerate(zip(self.estimators_, self.feature_indices_)):
        # 1. 找到当前树的袋外样本索引（总样本 - Bootstrap 采样索引）
        bootstrap_indices = self._bootstrap_sample(n_samples, i) # 复用采样逻辑
        all_indices = set(range(n_samples))
        bootstrap_indices_set = set(bootstrap_indices)
        oob_indices = list(all_indices - bootstrap_indices_set) # 袋外样本索引
        # 跳过无袋外样本的树（极端情况，如采样覆盖所有样本）
        if len(oob_indices) == 0:
            continue

```

```

# 2. 对袋外样本进行概率预测（使用当前树的特征子集）
X_oob = X[oob_indices][:, feature_indices]
oob_pred_proba = tree.predict_proba(X_oob) # 输出：[样本数×类别数]
# 3. 累加预测概率和计数（一个样本可能被多棵树的 OOB 集包含）
oob_predictions[oob_indices] += oob_pred_proba
oob_counts[oob_indices] += 1
# 2. 计算袋外分数（准确率）
# 筛选有 OOB 预测的样本（排除未被任何树预测的样本）
valid_indices = oob_counts > 0
if np.sum(valid_indices) == 0:
    self.oob_score_ = 0.0 # 无有效样本，分数为 0
    return
# 对有效样本，取概率最大的类别作为预测结果
final_predictions = np.argmax(oob_predictions[valid_indices], axis=1)
# 计算准确率（预测正确的样本数 / 有效样本数）
self.oob_score_ = np.mean(final_predictions == y[valid_indices])

```

（2）预测收集：每棵树仅对其袋外样本预测，累加所有树的预测概率（避免重复评估同一样本）

（3）分数计算：对每个样本，取“概率最大的类别”作为最终预测，与真实标签对比计算准确率（即 `oob_score_`）

2.3.5. 预测集成策略

随机森林通过“多棵树预测结果的集成”得到最终结果，核心是“概率平均”。

（1）概率平均集成：对测试样本，每棵树输出类别概率，取所有树的概率平均值作为最终概率，再通过“`.argmax`”得到预测类别：

```

def predict_proba(self, X):
    """预测类别概率：集成所有树的概率预测（平均法）"""
    X = np.asarray(X)
    n_samples = X.shape[0]
    # 初始化：存储所有树的预测概率（树数量×样本数×类别数）
    all_probas = []
    # 遍历每棵树，获取其预测概率
    for tree, feature_indices in zip(self.estimators_, self.feature_indices_):
        # 仅使用当前树训练时的特征子集（保持一致性）
        X_subset = X[:, feature_indices]
        # 获取当前树的预测概率（[样本数×类别数]）
        proba = tree.predict_proba(X_subset)
        all_probas.append(proba)
    # 集成：对所有树的概率取平均值（降低单棵树的随机性）
    avg_proba = np.mean(all_probas, axis=0)
    return avg_proba

def predict(self, X):
    """预测类别：取概率最大的类别（基于 predict_proba）"""

```

```
proba = self.predict_proba(X)
return np.argmax(proba, axis=1)
```

(2) 健壮性处理机制：针对“部分树的预测概率维度与总类别数不匹配”的极端情况（如某棵树的训练集未包含所有类别），添加形状调整逻辑：

```
def predict_proba(self, X):
    # （续上）在获取 proba 后添加健壮性处理
    proba = tree.predict_proba(X_subset)
    # 健壮性处理：确保概率数组维度与总类别数一致
    if proba.shape[1] != self.n_classes_:
        # 初始化调整后的概率数组（维度：样本数×总类别数）
        adjusted_proba = np.zeros((n_samples, self.n_classes_))
        # 取当前树的概率维度与总类别数的较小值，避免索引越界
        min_dim = min(proba.shape[1], self.n_classes_)
        # 将当前树的概率赋值到调整后的数组（缺失类别概率默认为 0）
        adjusted_proba[:, :min_dim] = proba[:, :min_dim]
        proba = adjusted_proba
    all_probas.append(proba)
    # （后续同前：计算平均概率）
```

2.3.6. 参数系统与调优兼容性

(1) 完整参数配置：支持随机森林的所有关键参数，可通过__init__灵活配置，适配不同数据集与场景。

```
def __init__(self,
              n_estimators=100,      # 基学习器（决策树）的数量（默认 100 棵）
              max_depth=None,        # 每棵树的最大深度（None 表示不限制，默认 None）
              max_features='auto',   # 每棵树分裂时考虑的特征数（默认取平方根）
              random_state=None,     # 随机种子（确保实验可复现，默认 None）
              bootstrap=True,        # 是否使用 Bootstrap 采样（默认 True，开启随机性）
              oob_score=False,       # 是否启用袋外评估（默认 False，关闭可节省计算）
              min_samples_split=2,    # 节点分裂所需的最小样本数（默认 2，防止过拟合）
              min_samples_leaf=1):    # 叶节点所需的最小样本数（默认 1，防止过拟合）

    # 初始化超参数
    self.n_estimators = n_estimators
    self.max_depth = max_depth
    self.max_features = max_features
    self.random_state = random_state
    self.bootstrap = bootstrap
    self.oob_score = oob_score
    self.min_samples_split = min_samples_split
    self.min_samples_leaf = min_samples_leaf

    # 初始化存储变量（训练后赋值）
    self.estimators_ = []            # 存储所有基学习器（决策树）
    self.feature_indices_ = []       # 存储每棵树使用的特征索引
```

```

self.classes_ = None          # 存储所有类别标签
self.n_classes_ = None       # 存储类别数量
self.oob_score_ = None       # 存储袋外评估分数（启用时赋值）

```

（2）网格搜索兼容性

①参数获取接口（get_params）

返回所有可配置超参数及其当前值，供调优工具读取参数空间。

```

def get_params(self, deep=True):
    """获取模型超参数（适配 scikit-learn 网格搜索）"""
    return {
        'n_estimators': self.n_estimators,
        'max_depth': self.max_depth,
        'max_features': self.max_features,
        'random_state': self.random_state,
        'bootstrap': self.bootstrap,
        'oob_score': self.oob_score,
        'min_samples_split': self.min_samples_split,
        'min_samples_leaf': self.min_samples_leaf
    }

```

②参数设置接口（set_params）

接收外部传入的参数键值对，动态更新模型超参数，供调优工具迭代参数组合。

```

def set_params(self, **parameters):
    """设置模型超参数（适配 scikit-learn 网格搜索）"""
    for parameter, value in parameters.items():
        # 动态为模型属性赋值（参数名需与__init__严格一致）
        setattr(self, parameter, value)
    return self # 返回更新后的模型实例，支持链式调用

```

2.4. models/custom_naivebayes.py

2.4.1. 高斯朴素贝叶斯实现

①高斯概率密度计算

通过高斯概率密度函数（PDF）计算单个特征的似然概率，包含方差平滑以避免零方差导致的计算错误：

```

def _gaussian_pdf(self, x, mean, var):
    """计算高斯概率密度函数（含方差平滑机制）"""
    var = np.maximum(var, self.var_smoothing) # 方差平滑：确保方差不小于 var_smoothing，防止除零
    exponent = np.exp(-((x - mean) ** 2) / (2 * var)) # 高斯 PDF 公式：f(x) = 1/√(2πσ²) * e^(-(x-μ)²/(2σ²))
    return (1 / np.sqrt(2 * np.pi * var)) * exponent

```

②高斯分布参数估计

遍历所有类别，计算每个类别下各特征的“均值（ μ ）”和“方差（ σ^2 ）”，作为正态分布的参数：

```

def _compute_gaussian_parameters(self, X, y):

```

```

"""计算高斯分布的核心参数（类别级均值与方差）"""
for i, c in enumerate(self.classes_):
    # 提取当前类别的所有样本
    X_c = X[y == c]
    # 计算该类别下各特征的均值（ $\mu$ ）和方差（ $\sigma^2$ ）
    self.theta_[i, :] = X_c.mean(axis=0) # theta_: 均值矩阵（类别×特征）
    self.sigma_[i, :] = X_c.var(axis=0) # sigma_: 方差矩阵（类别×特征）
    # 应用方差平滑，防止后续 PDF 计算中出现零方差
    self.sigma_[i, :] = np.maximum(self.sigma_[i, :], self.var_smoothing)

```

2. 4. 2. 多项式朴素贝叶斯实现

①多项式分布参数计算

统计每个类别下的特征总计数，结合拉普拉斯平滑计算特征条件概率的对数：

```

def _compute_multinomial_parameters(self, X, y):
    """计算多项式分布的核心参数（特征计数+对数概率）"""
    n_classes = len(self.classes_)
    n_features = X.shape[1]
    # 初始化类别级特征计数矩阵（维度：类别×特征）
    self.feature_count_ = np.zeros((n_classes, n_features))
    # 统计每个类别下各特征的总计数
    for i, c in enumerate(self.classes_):
        X_c = X[y == c] # 当前类别的所有样本
        self.feature_count_[i, :] = X_c.sum(axis=0) # 累加特征计数
    # 计算特征条件概率的对数（应用拉普拉斯平滑，避免零概率）
    feature_count_total = self.feature_count_.sum(axis=1, keepdims=True) # 每个类别的总特征数
    self.feature_log_prob_ = np.log(self.feature_count_ + self.alpha) - \
        np.log(feature_count_total + self.alpha * n_features)

```

②拉普拉斯平滑机制

```

# 拉普拉斯平滑公式：为特征计数和总计数添加平滑项，确保概率非零
probability = (feature_count + alpha) / (total_count + alpha * n_features)

```

2. 4. 3. 数值稳定性优化

①对数概率计算

将“概率连乘”转换为“对数概率求和”，避免小数连乘的下溢问题。以高斯分布为例：

```

def _joint_log_likelihood_gaussian(self, X):
    """计算高斯分布下的联合对数似然：log(P(X|y)×P(y))"""
    n_samples, n_features = X.shape
    n_classes = len(self.classes_)
    joint_log_likelihood = np.zeros((n_samples, n_classes))
    for i in range(n_classes):
        # 累加类别先验的对数：log(P(y))
        joint_log_likelihood[:, i] = np.log(self.class_prior_[i])

```

```

# 累加每个特征的对数似然:  $\log(P(X_1|y)) + \log(P(X_2|y)) + \dots$ 
for j in range(n_features):
    log_prob = np.log(self._gaussian_pdf(X[:, j], self.theta_[j], self.sigma_[j]))
    joint_log_likelihood[:, i] += log_prob
return joint_log_likelihood

```

②对数求和指数技巧

在计算后验概率归一化项时，使用 `np.logaddexp.reduce` 避免直接指数化导致的下溢，公式推导：

```

# 1. 计算所有类别的联合对数似然:  $\log(P(X|y) \times P(y))$ 
joint_log_likelihood = ... # 基于分布类型计算
# 2. 计算归一化项的对数:  $\log(P(X)) = \log(\sum_y \exp(\log(P(X|y) \times P(y))))$ 
log_prob_x = np.logaddexp.reduce(joint_log_likelihood, axis=1)
# 3. 计算后验概率:  $P(y|X) = \exp(\log(P(X|y) \times P(y)) - \log(P(X)))$ 
proba = np.exp(joint_log_likelihood - log_prob_x.reshape(-1, 1))

```

2.4.4. 参数验证

①完整性参数检查

在模型训练前验证超参数合法性，避免无效参数导致的运行错误：

```

def _check_params(self):
    """检查超参数有效性，抛出非法参数异常"""
    if self.distribution not in ['gaussian', 'multinomial']:
        raise ValueError("distribution must be 'gaussian' or 'multinomial'")
    if self.var_smoothing < 0:
        raise ValueError("var_smoothing must be non-negative")
    if self.alpha < 0:
        raise ValueError("alpha must be non-negative")

```

②数据合法性验证

针对多项式分布的特性，检查输入特征是否满足“非负”要求（离散计数特征不能为负）：

```

# 在 fit 方法中执行数据验证（仅多项式分布）
if self.distribution == 'multinomial' and np.any(X < 0):
    raise ValueError("Multinomial Naive Bayes requires non-negative feature values")

```

2.4.5. 网格搜索兼容性

①参数获取接口（get_params）

返回所有可配置超参数及其当前值，供调优工具读取参数空间：

```

def get_params(self, deep=True):
    """获取模型所有超参数（适配 scikit-learn 网格搜索）"""
    return {
        'priors': self.priors, # 自定义类别先验概率
        'var_smoothing': self.var_smoothing, # 高斯分布方差平滑项
        'distribution': self.distribution, # 特征分布假设
    }

```

```

'alpha': self.alpha,          # 多项式分布拉普拉斯平滑项
'fit_prior': self.fit_prior,   # 是否从数据学习先验概率
'random_state': self.random_state # 随机种子（确保可复现）
}

```

②参数设置接口（set_params）

接收外部传入的参数键值对，动态更新模型超参数，供调优工具迭代参数：

```

def set_params(self, **parameters):
    """设置模型超参数（适配 scikit-learn 网格搜索）"""
    for parameter, value in parameters.items():
        # 动态为模型属性赋值（参数名需与__init__方法一致）
        setattr(self, parameter, value)
    return self # 返回更新后的模型实例

```

3. 代码运行

3.1. 对所有算法进行测试，并采用随机划分的数据集以验证代码的正确性。

```

(DM) lsx@escenter55-SYS-7049GP-TRT:~/code/DM-Group-Project$ python run.py --algorithm custom_naivebayes --dataset autos
第1折交叉验证准确率: 0.4390
第2折交叉验证准确率: 0.4878
第3折交叉验证准确率: 0.4634
第4折交叉验证准确率: 0.5122
第5折交叉验证准确率: 0.4390
交叉验证准确率: ['0.4390', '0.4878', '0.4634', '0.5122', '0.4390']
平均交叉验证准确率: 0.4683 (±0.0284)
混淆矩阵已保存到: result/custom_naivebayes/confusion_matrix_custom_naivebayes_autos_20250929_000349.png
结果已保存到: result/custom_naivebayes/custom_naivebayes_autos_20250929_000350.txt 和 result/custom_naivebayes/custom_naivebayes_autos_20250929_000350.json
交叉验证准确率图表已保存到: result/custom_naivebayes/cross_val_accuracy_custom_naivebayes_autos_20250929_000350.png
结果已汇总到: result/summary.csv

分类报告:

```

	precision	recall	f1-score	support
0	0.50	1.00	0.67	5
1	1.00	1.00	1.00	1
2	1.00	0.11	0.20	18
3	0.56	0.62	0.59	8
4	0.09	0.20	0.12	5
5	0.50	1.00	0.67	4
accuracy			0.44	41
macro avg	0.61	0.66	0.54	41
weighted avg	0.69	0.44	0.39	41

3.2. 为自定义的 K 近邻 (KNN)、随机森林 (Random Forest) 和朴素贝叶斯 (Naive Bayes) 算法设置合理的网格搜索调参范围，并运行程序输出结果。

(1) 当算法为 custom_knn 时，调参范围如下：

```

{
    'n_neighbors': knn_neighbors,
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}

```

(2) 当算法为 custom_randomforest 时，调参范围如下：

```

{
    'n_estimators': n_estimators,
    'max_depth': max_depth,
}

```

```
'min_samples_split': [2, 5],
'max_features': ['auto', 'sqrt'],
'oob_score': [True, False]
}
```

(3) 当算法为 custom_naivebayes 时，调参范围如下：

```
{
    'distribution': ['gaussian', 'multinomial'],
    'var_smoothing': [1e-9, 1e-8, 1e-7], # 仅适用于高斯分布
    'alpha': [0.1, 0.5, 1.0],           # 仅适用于多项式分布（拉普拉斯平滑）
    'fit_prior': [True, None]            # 是否学习类别先验
}
```

七、实验结果

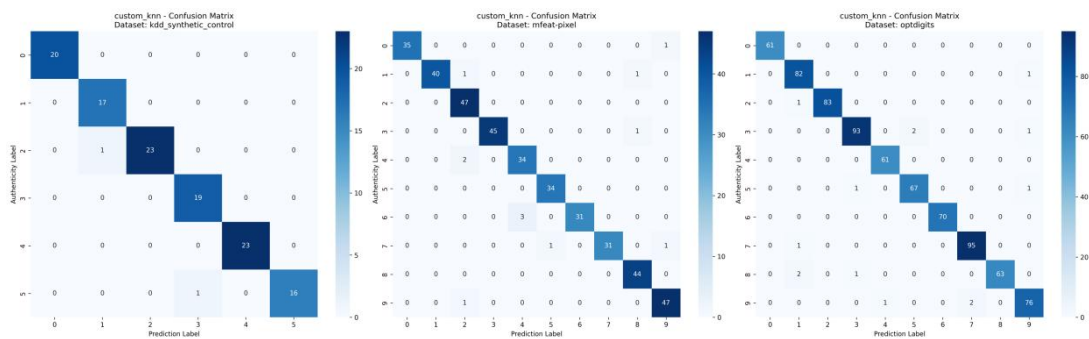
自定义 KNN 算法在各数据集上的参数调优结果					
数据集	algorithm	metric	n_neighbors	p'	weights
kdd_synthetic_control	brute	euclidean	3	1	uniform
mfeat-pixel	brute	manhattan	15	1	uniform
optdigits	brute	manhattan	7	1	distance
自定义 KNN 算法在各数据集上评估结果					
数据集	组号	精度	召回率	F1 分数	总体准确率
kdd_synthetic_control	0	1.0000	1.0000	1.0000	0.9833
	1	0.9444	1.0000	0.9714	
	2	1.0000	0.9583	0.9787	
	3	0.9500	1.0000	0.9744	
	4	1.0000	1.0000	1.0000	
	5	1.0000	0.9412	0.9697	
mfeat-pixel	0	1.0000	0.9722	0.9859	0.9700
	1	1.0000	0.9524	0.9756	
	2	0.9216	1.0000	0.9592	
	3	1.0000	0.9783	0.9890	
	4	0.9189	0.9444	0.9315	
	5	0.9714	1.0000	0.9855	
	6	1.0000	0.9118	0.9538	
	7	1.0000	0.9394	0.9688	
	8	0.9565	1.0000	0.9778	
optdigits	9	0.9592	0.9792	0.9691	0.9817
	0	1.0000	1.0000	1.0000	
	1	0.9535	0.9880	0.9704	
	2	1.0000	0.9881	0.9940	
	3	0.9789	0.9688	0.9738	

	4	0.9839	1.0000	0.9919
	5	0.9710	0.9710	0.9710
	6	1.0000	1.0000	1.0000
	7	0.9794	0.9896	0.9845
	8	1.0000	0.9545	0.9767
	9	0.9620	0.9620	0.9620

自定义 KNN 算法在各数据集上 5 折交叉验证的结果

数据集	准确率					
	第一折	第二折	第三折	第四折	第五折	平均
kdd_synthetic_control	0.9833	1.0000	1.0000	1.0000	0.9917	0.9950 (±0.0067)
mfeat-pixel	0.9700	0.9875	0.9750	0.9675	0.9775	0.9755 (±0.0070)
optdigits	0.9817	0.9817	0.9739	0.9673	0.9856	0.9780 (±0.0066)

自定义 KNN 算法在各数据集上的混淆矩阵评估



自定义 Randomforest 算法在各数据集上的参数调优结果

数据集	min_samples_split	max_depth	max_features	n_estimators	oob_score
kdd_synthetic_control	5	None	auto	100	True
mfeat-pixel	2	25	auto	150	False
optdigits	2	None	auto	150	True

自定义 Randomforest 算法在各数据集上评估结果

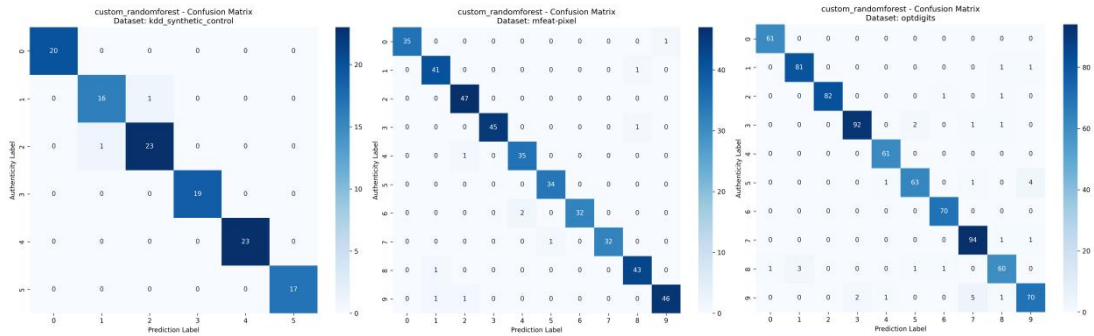
数据集	组号	精度	召回率	F1 分数	总体准确率	袋外分数
kdd_synthetic_co	0	1.0000	1.0000	1.0000	0.9833	0.9896
	1	0.9412	0.9412	0.9412		
	2	0.9583	0.9583	0.9583		
	3	1.0000	1.0000	1.0000		
	4	1.0000	1.0000	1.0000		
mfeat-pixel	5	1.0000	1.0000	1.0000	0.9750	\
	0	1.0000	0.9722	0.9859		
	1	0.9535	0.9762	0.9647		
	2	0.9592	1.0000	0.9792		
	3	1.0000	0.9783	0.9890		
	4	0.9459	0.9722	0.9589		
	5	0.9714	1.0000	0.9855		
	6	1.0000	0.9412	0.9697		
	7	1.0000	0.9697	0.9846		

optdigits	8	0.9556	0.9773	0.9663	0.9595	0.9424
	9	0.9787	0.9583	0.9684		
	0	0.9839	1.0000	0.9919		
	1	0.9643	0.9759	0.9701		
	2	1.0000	0.9762	0.9800		
	3	0.9787	0.9583	0.9684		
	4	0.9683	1.0000	0.9839		
	5	0.9545	0.9130	0.9333		
	6	0.9722	1.0000	0.9859		
	7	0.9307	0.9792	0.9543		
	8	0.9231	0.9091	0.9160		
	9	0.9211	0.8861	0.9032		

自定义 Randomforest 算法在各数据集上 5 折交叉验证的结果

数据集	准确率					
	第一折	第二折	第三折	第四折	第五折	平均
kdd_synthetic_control	0.9917	0.9833	1.0000	1.0000	1.0000	0.9950 (±0.0067)
mfeat-pixel	0.9675	0.9800	0.9725	0.9700	0.9800	0.9740 (±0.0051)
optdigits	0.9582	0.9608	0.9503	0.9516	0.9673	0.9576 (±0.0062)

自定义 Randomforest 算法在各数据集上的混淆矩阵评估



自定义 Naivebayes 算法在各数据集上的参数调优结果

数据集	alpha	distribution	fit_prior'	var_smoothing
kdd_synthetic_control	0.1	gaussian	True	1e-09
mfeat-pixel	0.1	gaussian	True	1e-07
optdigits	0.5	multinomial	True	1e-09

自定义 Naivebayes 算法在各数据集上评估结果

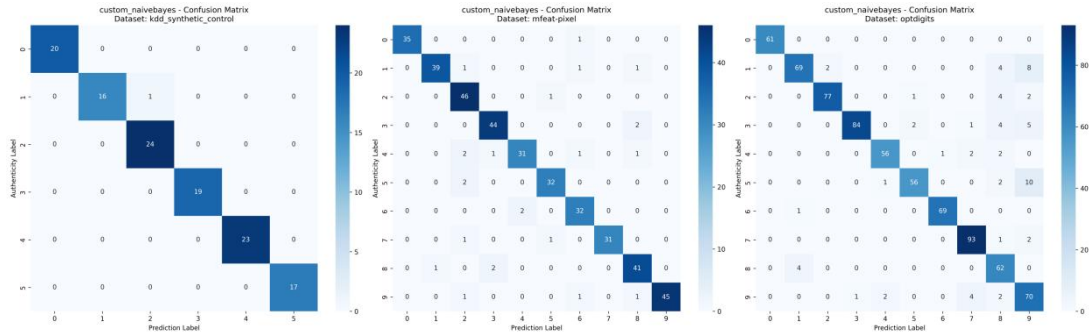
数据集	组号	精度	召回率	F1 分数	总体准确率
kdd_synthetic_co ntrol	0	1.0000	1.0000	1.0000	0.9917
	1	1.0000	0.9412	0.9697	
	2	0.9600	1.0000	0.9796	
	3	1.0000	1.0000	1.0000	
	4	1.0000	1.0000	1.0000	
mfeat-pixel	5	1.0000	1.0000	1.0000	0.9400
	0	1.0000	0.9722	0.9859	
	1	0.9750	0.9286	0.9512	

	2	0.8679	0.9787	0.9200	
	3	0.9362	0.9565	0.9462	
	4	0.9394	0.8611	0.8986	
	5	0.9412	0.9412	0.9412	
	6	0.8889	0.9412	0.9143	
	7	1.0000	0.9394	0.9688	
	8	0.8913	0.9318	0.9111	
	9	1.0000	0.9375	0.9677	
	0	1.0000	1.0000	1.0000	
	1	0.9324	0.8313	0.8790	
optdigits	2	0.9747	0.9167	0.9448	0.9111
	3	0.9882	0.8750	0.9282	
	4	0.9492	0.9180	0.9333	
	5	0.9492	0.8116	0.8750	
	6	0.9857	0.9857	0.9857	
	7	0.9300	0.9688	0.9490	
	8	0.7654	0.9394	0.8435	
	9	0.7216	0.8861	0.7955	

自定义 Naivebayes 算法在各数据集上 5 折交叉验证的结果

数据集	准确率					
	第一折	第二折	第三折	第四折	第五折	平均
kdd_synthetic_control	0.9917	1.0000	1.0000	1.0000	0.9917	0.9967 (±0.0041)
mfeat-pixel	0.9400	0.9200	0.9050	0.9150	0.9175	0.9195 (±0.0114)
optdigits	0.9111	0.9085	0.9124	0.9097	0.9398	0.9163 (±0.0118)

自定义 Naivebayes 算法在各数据集上的混淆矩阵评估



八、实验问题以及解决过程

1. 在数据预处理阶段，针对部分数据集中存在大量缺失值等问题，我们在特征选择环节进行了相应处理：删除了缺失值占比过高的特征，同时采用 SelectKBest 方法动态筛选信息量丰富的特征，以提升输入数据的质量与模型效果。
2. 在设置算法参数时，由于超参数组合空间庞大，难以凭经验预先确定最优配置。为此，我们引入 scikit-learn 中的网格搜索方法，对参数空间进行系统寻优，

以获取近似最优的超参数组合。

3. 实验中发现，自定义实现的随机森林算法在运行速度上显著慢于 scikit-learn 提供的版本。查阅相关资料，我们发现 scikit-learn 在底层实现中采用了更高效的特征排序与分裂点选择策略，支持多线程并行计算与增量学习，同时在数值稳定性、缺失值处理及内存管理等方面也进行了深度优化，从而大幅提升了执行效率。

九、实验小结

本次数据挖掘实验围绕分类问题展开，重点完成了 K 近邻、朴素贝叶斯与随机森林三种经典算法的从零实现，并在三个公开数据集上进行了验证。通过将自实现模型与 scikit-learn 官方实现进行对比，我们不仅验证了算法结果的正确性，也深入分析了两者在执行效率上的显著差异，从而更全面地评估了自实现模型的性能表现。实验过程中，我们系统探索了关键超参数对分类性能的影响，并借助网格搜索技术实现了自动化参数调优；针对数据质量不佳、特征维度高等现实问题，采用了特征筛选与缺失值处理等策略，有效提升了模型的鲁棒性与泛化能力。最终，所有自实现模型均在测试集上取得了优良性能，验证了算法实现的正确性，同时通过模块化的代码重构，为后续算法迭代与工程应用奠定了良好基础。

十、人员及分工

成员	分工
林圣翔	负责项目统筹、整体代码架构设计、Random Forest 算法的实现、算法评估、实验报告撰写等。
易怀川	负责数据预处理、KNN 算法实现、超参数自动调优代码的开发、实验报告撰写等。
李彦锟	负责 Naive Bayes 算法实现，并完成实验报告的整理与汇总工作。